

Development of algorithms for 3D modeling of scars following orthopedic surgery

Bastien Roure

Supervised by : Marek Bucki

Abstract—This paper gives an overview of the different algorithms implemented in **C++** for modeling post-operative scars in orthopedic surgery for the company Twinsight Medical. The aim of this work is, starting from a **digital twin** of a patient's knee modeled by a **3D mesh**, to approximate the **geodesic** connecting two points previously placed on this mesh. **Dijkstra's shortest path** and **A* algorithm** are a first approximation of this geodesic, which will later be improved with **different algorithms**. The resulting path will then be the area where the scar is created by a **Gaussian deformation** of the mesh, after a **densification**. This work provides a practical approach to algorithmic modeling of post-operative scarring on complex surfaces. It provides an efficient and reasonably accurate visual estimate of the knee's appearance after surgery.

Index Terms—Geodesic computation, Dijkstra algorithm, A* algorithm, 3D mesh, Mesh deformation, Post-operative scar modeling, Digital twin, Orthopedic surgery, C++ implementation, Medical visualization.

1 INTRODUCTION

When preparing for orthopedic surgery, it's important for the doctor to explain the details of the operation to the patient, so that he understands what's going to happen to the body, and prepares psychologically for the potential repercussions. However, one detail that is sometimes overlooked is the aesthetic result of the surgeon's intervention. When an incision is made, the result is a scar that will often be the patient's partner for many years to come. This mark, which tells stories, can sometimes have a major psychological impact. It's crucial to be able to show the patient beforehand what his knee could potentially look like after the operation.

To best suit the individual, this scar can be placed on the patient's digital knee twin, which in this work is supplied in advance by the **Twinsight Medical** company. This object is a faithful 3D computer representation of the knee's appearance, enabling a customized solution for each person.

The aim of this internship is, starting from the 3D object that is the patient's knee, to approximate the shortest path along this model, between two points previously placed on it. This distance is commonly called **geodesic**. To approximate this so-called geodesic, we first need to represent the object as a graph to apply **Dijkstra's** and **A*'s** algorithms. Then we'll look at a first approach using equation solving to give us a starting point for using an **iterative local length minimization algorithm**. Once the geodesic has been approximated, we have the outline of our scar, and all we need to do is re-densify the mesh zone around it to allow smooth **Gaussian deformation**, giving us a result that comes close to the appearance of a real scar.

This report presents the methodology developed to compute these geodesic approximations, as well as the trade-offs involved in using discrete surface representations. Particular emphasis is placed on balancing computational efficiency and geometric accuracy, as well as on integrating the resulting scar path into a deformation pipeline for realistic visualization. This work thus contributes to a practical approach to improving patient psychological comfort in anticipation of surgery.

2 BACKGROUND

2.1 3D Objects

As introduced earlier, in this internship all work is done on 3D objects Figure 1. In computer graphics, the most common way of representing surfaces is with a polygonal mesh, and more precisely here with a triangular mesh. This consists of **vertices** connected by **edges**, forming a set of triangles. This approach makes it possible to discretize a continuous surface into a finite set of simple elements. This flexible, efficient representation can be used for modeling, 3D animation and complex physical simulations.

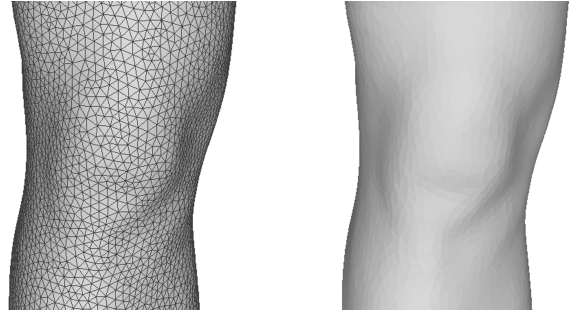


Fig. 1. Example of 3D Knee used during this internship, with and without all the points displayed.

Formally, a **triangular mesh** $\mathcal{M}$ is defined as follows :

$$\mathcal{M} = (V, E, F) \quad (1)$$

With $V = \{v_1, v_2, \dots, v_n\} \subset \mathbb{R}^3$: the set of vertices being points in space; $E \subset V \times V$: the set of edges connecting vertices in pairs; $F \subset V \times V \times V$: the set of triangular faces defined by a triplet of vertices (often ordered counter-clockwise to represent orientation).

Topological constraints are imposed: each edge must belong to a face, and each face must be composed of elements existing in E .

This representation of $\mathcal{M}$ can also be seen in the form of a non-oriented weighted graph $G = (V, E)$, where V graph nodes correspond to mesh points and E graph edges correspond to mesh edges, weighted by their distance.

This abstraction makes it possible to run algorithms on **non-oriented weighted graphs** subsection 3.1 and subsection 3.2.

2.2 Geodesic

2.2.1 Geodesic on a Mesh

In a triangular $\mathcal{M}$ mesh, defined Equation 1, the surface is discretized, so a discrete geodesic is required. The idea is to calculate the shortest path between two points on the mesh that respects the surface structure.

One approach may be to calculate the geodesic along the edges via a graph representation where each edge $(v_i, v_j) \in E$ is weighted by its Euclidean length. Then it's just a matter of calculating the path that minimizes the cost between two vertices. This method is a rough approximation, based solely on the edges of the mesh.

Another approach is to create a path of triangles on this surface, in order to optimize the layout and achieve a better result. This technique will be used during this internship (see section 3).

3 FOUNDATIONS OF THE WORK

As introduced earlier, we are using a graph-based representation to compute the path of triangles connecting two points in the shortest possible way.

To do so, we define $G^* = (\mathcal{V}, \mathcal{E})$, a slightly different variant graph. Here $\mathcal{V}$ is the set of triangle centroids¹, one per face of the mesh, and $\mathcal{E}$ connects two centroids if the corresponding triangles in the original mesh are adjacent, if they share a common edge.

In other words, there exists an edge $e_1 \in E$ linking $(v_i, v_j) \in V$ if and only if triangles Tr_i and Tr_j share an edge in the original mesh.

For any edge $e \in \mathcal{E}$ of the graph G^* , the weight is computed along the mesh surface as follows. Let $(v_i, v_j) \in \mathcal{V}$ be the centroids of two adjacent triangles Tr_i and Tr_j , and let $adj_{i,j} \subset \mathbb{R}^3$ denote the edge shared by both triangles, with $a, b \in \mathbb{R}^3$ its endpoints.

The weight $w(e)$ is then defined as:

$$w(e) = \|v_i - m_{i,j}\| + \|m_{i,j} - v_j\| \quad \text{where} \quad m_{i,j} = \frac{1}{2}(a + b)$$

This formulation ensures that the path better follows the surface curvature while remaining within the respective triangles, unlike a direct centroid-to-centroid connection which may cut through the mesh in an unrealistic way.

We now have a non-oriented weighted graph G^* , to which we can apply shortest-path algorithms.

3.1 Dijkstra Algorithm

One of the best-known algorithms for finding the shortest paths in a graph is the Dijkstra algorithm [1].

The main idea is to progressively explore the nodes of the graph, keeping the minimum distance from its predecessor for each node. This distance is updated as shorter paths are discovered. Figure 2

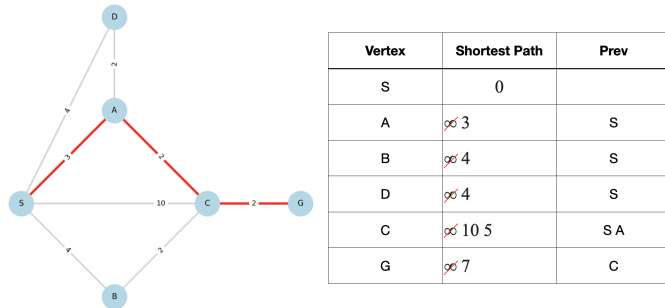


Fig. 2. A small example of the result of Dijkstra's algorithm. Here, all nodes have been visited (**G**, the goal, being the last in the algorithm). For each node, we can reconstruct the shortest path. To go from **S** to **G**, the path is : **[S, A, C, G]**.

In our case, the algorithm starts from the centroid of the triangle containing the starting point of the scar. At initialization, all nodes receive an infinite distance, except the start node, whose distance is set to zero. A priority queue is used to always give priority to the node with the smallest current distance.

At each iteration, the node with the smallest distance is extracted, then its adjacent neighbors are examined. If a shorter

1. The centroid of a triangle is the average of its three vertex coordinates.

path to a neighbor is found via the current node, its distance is updated, and the node is reinserted into the priority queue with this new distance and its predecessor allowing it to be reached. The process repeats until the destination node is attained.

Once it is achieved, the minimal path can be retraced by following the stored predecessors, enabling the optimal path from the source to be reconstructed.

Here is Dijkstra's pseudo-code for shortest path :

Algorithm 1 Dijkstra's Algorithm

Require: Graph $G^* = (\mathcal{V}, \mathcal{E})$ with non-negative edge weights, source node $s \in \mathcal{V}$, target node $t \in \mathcal{V}$

```

1: for all  $v \in \mathcal{V}$  do
2:    $distance[v] \leftarrow \infty$ 
3:    $previous[v] \leftarrow \text{null}$ 
4: end for

5:  $distance[s] \leftarrow 0$ 
6:  $Q \leftarrow$  priority queue initialized with all  $v \in \mathcal{V}$ 

7: while  $Q$  is not empty do
8:    $u \leftarrow \text{ExtractMin}(Q)$ 
9:   if  $u = t$  then
10:    break {Target reached}
11:   end if
12:   for all  $v$  neighbor of  $u$  do
13:      $dist \leftarrow distance[u] + weight(u, v)$ 
14:     if  $dist < distance[v]$  then
15:        $distance[v] \leftarrow dist$ 
16:        $previous[v] \leftarrow u$ 
17:        $\text{UpdatePriority}(Q, v, dist)$ 
18:     end if
19:   end for
20: end while
21:
22: return shortest path by tracing back from  $t$  using  $previous$ 

```

3.2 A* algorithm

Another algorithm evaluated for computing the shortest path is a heuristic extension of Dijkstra's algorithm, known as A* [2]. By introducing an heuristic that estimates the remaining distance to the target, A* can significantly reduce the number of nodes explored while still guaranteeing the optimal path Figure 3.

In this algorithm, heuristics are a means of sorting the nodes to be visited. Using as a score :

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the score of the path from the start and $h(n)$ is the heuristic (generally Euclidean distance) to the goal. This method allows you to keep an idea of how to handle summits in terms of getting closer to the destination if there is a good compromise between the weight of the ridge and the distance to the point. This usually reduces the number of vertices visited by Dijkstra's algorithm.

Here is the pseudo-code of the A* algorithm :

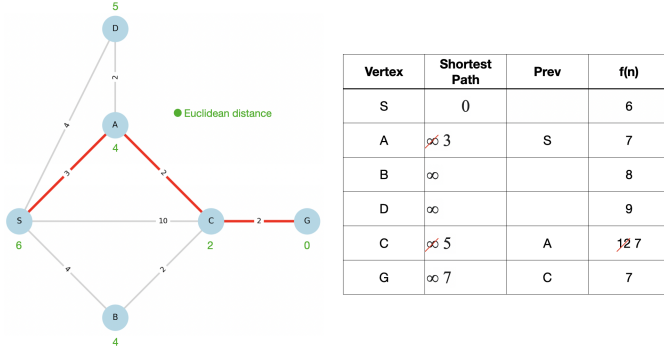


Fig. 3. Using the same simple example as for Dijkstra, to go from the starting point **S** to the destination **G**, we only visit the nodes: [**S**, **A**, **C**, **G**]. This large-scale gain can be a big time saver, but this is not always the case.

Algorithm 2 A* Algorithm

Require: Graph $G = (\mathcal{V}, E)$ with non-negative edge weights, source node $s \in \mathcal{V}$, target node $t \in \mathcal{V}$, admissible heuristic $h(v)$

```

1: for all  $v \in \mathcal{V}$  do
2:    $g[v] \leftarrow \infty$  {Cost from  $s$  to  $v$ }
3:    $f[v] \leftarrow \infty$  {Estimated total cost  $g[v] + h(v)$ }
4:    $previous[v] \leftarrow \text{null}$ 
5: end for

6:  $g[s] \leftarrow 0$ 
7:  $f[s] \leftarrow h(s)$ 
8:  $Q \leftarrow$  priority queue initialized with  $(s, f[s])$ 

9: while  $Q$  is not empty do
10:   $u \leftarrow \text{ExtractMin}(Q)$  {Node with smallest  $f$ }
11:  if  $u = t$  then
12:    break {Target reached}
13:  end if
14:  for all  $v$  neighbor of  $u$  do
15:     $tentative_g \leftarrow g[u] + \text{weight}(u, v)$ 
16:    if  $tentative_g < g[v]$  then
17:       $g[v] \leftarrow tentative_g$ 
18:       $f[v] \leftarrow g[v] + h(v)$ 
19:       $previous[v] \leftarrow u$ 
20:       $\text{UpdatePriority}(Q, v, f[v])$ 
21:    end if
22:  end for
23: end while

```

3.3 First Geodesic Approximation

Using the triangular path calculated previously, we obtain a first approximation of the scar by connecting the centroids of the triangles along the path, using the midpoints of the edges they share. More formally, the path is defined as follows : given a start point S and an endpoint T , we have a sequence of segments $[U_i, V_i]$, each representing an edge shared by two adjacent triangles in the path Figure 4.

The idea is then to place a point p_i on each edge $[U_i, V_i]$, such that the total length is formed by the sum of the segments $[p_i, p_{i+1}]$ for all $i \in \{1, 2, \dots, n-1\}$.

Each point p_i lies on its corresponding edge and is defined using a barycentric parameter $\alpha_i \in [0, 1]$, such that :

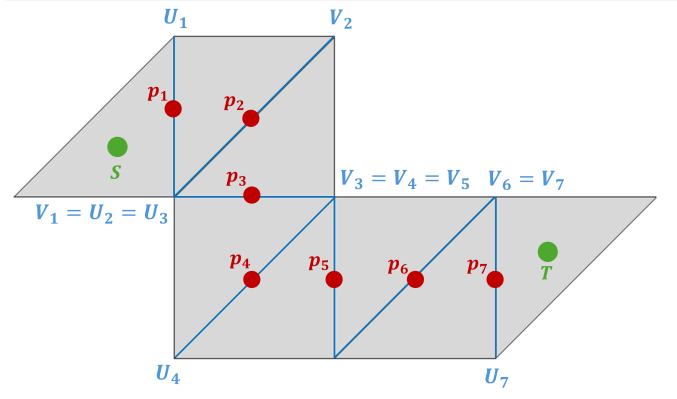


Fig. 4. An example of a triangle path is shown, where the start and end points are shown in green, the adjacent edges $[U_i, V_i]$ are highlighted in blue and the points p_i in red ($\alpha_i = 0.5$). **Note :** Although some edges may share a common point, these are treated as separate entities in our calculations for clarity.

$$p_i = U_i \cdot \alpha_i + (1 - \alpha_i) \cdot V_i$$

Using the Euclidean square distance (to simplify calculations), the total segment S_{tot} is expressed as :

$$S_{tot} = \sum_{i=0}^{n-1} \|p_i - p_{i+1}\|_2^2 = \sum_{i=0}^{n-1} (p_i - p_{i+1})^2$$

In this expression p_0 and p_n are respectively S and T .

The problem can therefore be summarized as placing the α_i , for all $i \in [1, n-1]$, in such a way as to minimize S_{tot} . So :

$$\min_{\alpha_i \in [0, 1]} \sum_{i=0}^{n-1} \|\alpha_i U_i + (1 - \alpha_i) V_i - (\alpha_{i+1} U_{i+1} + (1 - \alpha_{i+1}) V_{i+1})\|_2^2$$

To minimize the function S_{tot} with respect to the parameters α_i , we use a classic optimization method: we calculate the **gradient** of S_{tot} with respect to α_i , then we solve the resulting equation by setting this gradient to zero.

Indeed, a necessary condition for S_{tot} to reach a minimum is that all partial derivatives of S_{tot} with respect to the variables α_i be zero, in other words :

$$\frac{\partial S_{tot}}{\partial \alpha_i} = 0 \quad \text{for all } i \in \{1, \dots, n-1\}.$$

This allows us to find the stationary points of the function, among which we identify the global minimum. We then simply need to solve the following linear system :

$$\nabla S_{tot} = A\alpha + b = 0 \iff \alpha = -A^{-1} \cdot b \quad (2)$$

Where $\alpha \in [0, 1]^{n-1}$

The results obtained Figure 5 with this initial approach give a curve in red that appears to be close to the geodesic. However, the approximation can still be improved, as the use of the squared Euclidean distance introduces a bias. A new approach must therefore be used to achieve our objective.

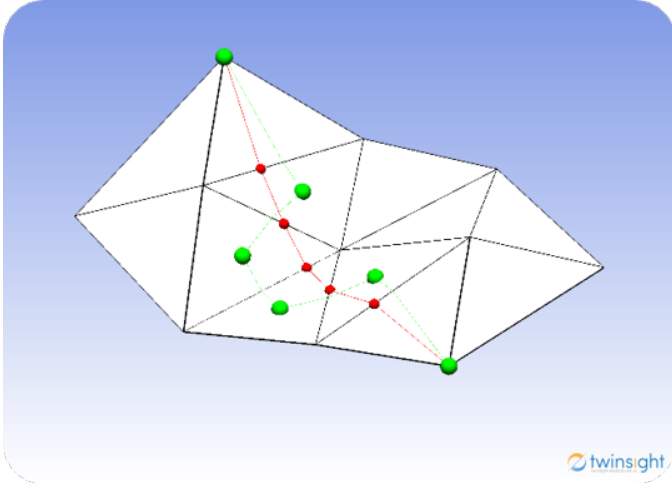


Fig. 5. 3D Toy Test where the green line represents Dijkstra's shortest path and the red line represents the path after solving Equation 2.

3.4 Iterative Algorithm

The idea behind this new algorithm is to straighten the curve found in the previous algorithm. Starting from this, we want to move the points p_i by calculating the intersection between the segments $[p_{i-1}, p_{i+1}]$ and $[U_i, V_i]$ Figure 6. This update rule can be interpreted as a first-order approximation of a gradient descent step, aimed at minimizing the total energy S_{tot} . Although it does not explicitly calculate the gradient, the displacement of each point p_i towards the intersection between $[p_{i-1}, p_{i+1}]$ and the edge $[U_i, V_i]$ reduces the local curvature, thus approximating the behavior of a gradient descent. To ensure convergence and avoid infinite oscillations due to small updates, a stopping criterion based on displacement tolerance is used. The algorithm stops when the amplitude of all updates (δ) falls below a predefined threshold.

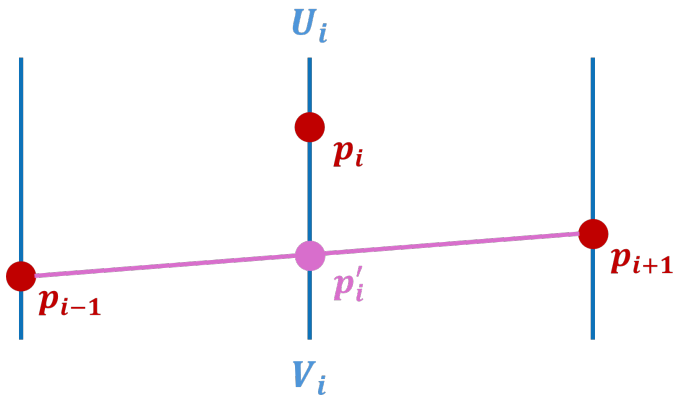


Fig. 6. Illustration of the calculation of the new position p'_i by calculating the intersection between $[p_{i-1}, p_{i+1}]$ and $[U_i, V_i]$

Let's start with a simple case :

3.4.1 2D case :

In this case, our two segments are coplanar and intersect. All that remains is to calculate at which point. We simply need to find the barycentric coefficients $s, t \in [0, 1]$ such that :

$$\begin{aligned} p_{i-1} + (p_{i+1} - p_{i-1}) \cdot s &= U_1 + (V_1 - U_1) \cdot t \\ \Leftrightarrow p_{i-1} + (p_{i+1} - p_{i-1}) \cdot s - [U_1 + (V_1 - U_1) \cdot t] &= 0 \end{aligned}$$

This calculation is equivalent to solving the equation :

$$\begin{aligned} \begin{bmatrix} p_{i+1}^x - p_{i-1}^x & U_1^x - V_1^x \\ p_{i+1}^y - p_{i-1}^y & U_1^y - V_1^y \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} &= \begin{bmatrix} U_1^x - p_{i-1}^x \\ U_1^y - p_{i-1}^y \end{bmatrix} \\ \Leftrightarrow \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} &= \begin{bmatrix} e \\ f \end{bmatrix} \\ \Leftrightarrow \begin{bmatrix} s \\ t \end{bmatrix} &= \frac{1}{ad - bc} \begin{bmatrix} de - bf \\ -ce + af \end{bmatrix} \end{aligned} \quad (3)$$

This resolution therefore allows us to place the intersection point in our basic 2-dimensional case. Given that we are working on a 3D surface, we must then find a way to calculate this intersection point when the two segments are not coplanar.

3.4.2 3D case :

Still with the idea of keeping a minimum distance when calculating the intersection of our two segments, in the 3D case we must first rotate one of our two triangles along the adjacent edge.

Let's take two triangles, defined by 4 points A, B, U, and V. The goal is to rotate point B along the axis [U, V] so that B', the projected point, ends up on the plane [A, U, V] Figure 7.

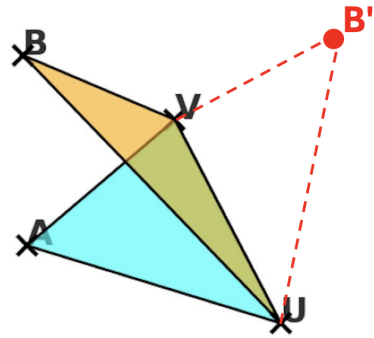


Fig. 7. Illustration of the two non-coplanar triangles with B' being the image of B after rotation along the axis [U, V].

First, we calculate the angle formed by our two triangles by taking the dot product between the vectors $\vec{AU}$ and $\vec{BU}$, which gives us the following calculation :

$$\vec{n}_1 = \vec{UV} \times \vec{UA}, \quad \vec{n}_2 = \vec{UV} \times \vec{UB}, \quad \theta = \cos^{-1} \left(\frac{\vec{n}_1 \cdot \vec{n}_2}{\|\vec{n}_1\| \|\vec{n}_2\|} \right)$$

Note : To ensure rotation in the correct direction, simply evaluate the sign of the following product :

$$\text{sgn} = \vec{UV} \cdot (\vec{n}_1 \times \vec{n}_2)$$

To have a counterclockwise rotation $\theta = -\theta$ if $\text{sgn} \leq 0$

The rotation matrix (Rot) can then be expressed as follows:

$$\begin{bmatrix} \cos \theta + k_x^2(1 - \cos \theta) & k_x k_y(1 - \cos \theta) - k_z \sin \theta & k_x k_z(1 - \cos \theta) + k_y \sin \theta \\ k_y k_x(1 - \cos \theta) + k_z \sin \theta & \cos \theta + k_y^2(1 - \cos \theta) & k_y k_z(1 - \cos \theta) - k_x \sin \theta \\ k_x k_z(1 - \cos \theta) - k_y \sin \theta & k_y k_z(1 - \cos \theta) - k_x \sin \theta & \cos \theta + k_z^2(1 - \cos \theta) \end{bmatrix}$$

After rotation, point B becomes B' according to the formula :

$$B' = \text{Rot} \cdot \overrightarrow{UB} + U$$

Now that our four points are on the same plane, we just need to calculate the intersection point as before. This time, the calculation is slightly different because a third dimension comes into play.

By rewriting the equation 3, we have :

$$\begin{bmatrix} B^x - A^x & U^x - V^x \\ B^y - A^y & U^y - V^y \\ B^z - A^z & U^z - V^z \end{bmatrix} \begin{bmatrix} s \\ t \end{bmatrix} = \begin{bmatrix} U^x - A^x \\ U^y - A^y \\ U^z - A^z \end{bmatrix} \quad (4)$$

We are then faced with a system that is said to be overconstrained. We must therefore approximate the result by minimizing the quadratic error :

$$\min_x \|Mx - y\|^2$$

The objective function can therefore be written as:

$$f(x) = \|Mx - y\|^2 = (Mx - y)^T (Mx - y)$$

In developing, we have :

$$f(x) = x^T M^T Mx - x^T M^T y - y^T Mx + y^T y$$

$$f(x) = x^T M^T Mx - 2x^T M^T y + y^T y$$

We derive with respect to x :

$$\nabla_x f(x) = 2M^T Mx - 2M^T y$$

By canceling the gradient :

$$M^T Mx = M^T y$$

As we have $M^T M$ invertible :

$$x = (M^T M)^{-1} M^T y$$

We said earlier that we are looking to approximate x , which would be problematic because x represents the values of S and T that allow us to find our intersection point. Ultimately, this is not a concern because, by construction with the previous algorithm, there is indeed an intersection point between $[A, B]$ and $[U, V]$ since they are now coplanar.

This algorithm therefore allows us to approach our geodesic iteratively and enables us to obtain the outline of our scar on our 3D surface Figure 8.

3.5 Gaussian Mesh Deformation

Now that we have a reasonable approximation of the desired geodesic to model our scar, we must now work on its graphic appearance so that the visual rendering is clear to the patient.

To do this, we first need to clear a path of triangles along our final outline. This path must be wide enough for the scar to be graphically realistic, but light enough so that the calculation time during remeshing is not too long. This remeshing step is crucial because a triangular mesh composed of large faces results in a rough approximation of shapes, which detracts from the realism of the rendering. We must therefore increase

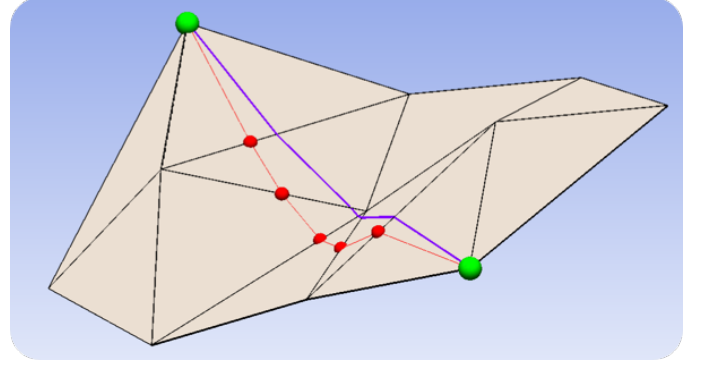


Fig. 8. Toy test comparing the previous algorithm (red) with the iterative algorithm (purple).

the number of triangles in this area close to the geodesic by multiplying the number of triangles on it.

We perform the remeshing as follows : for each triangle, we take the center of each edge and connect these points to form a new central triangle that divides our initial triangle into four triangles (see Figure 9). This means that we can calculate the number of triangles that will result after this operation as follows :

$$N_t = n \cdot 4^k$$

With N_t the final number of triangles, n the number of triangles at the beginning, and k the number of times we do the remeshing. It is therefore important to strike a balance between the quality of the triangles and the number of triangles in the mesh, as the latter will increase exponentially, which could become too costly in terms of computation for the rest of the process.

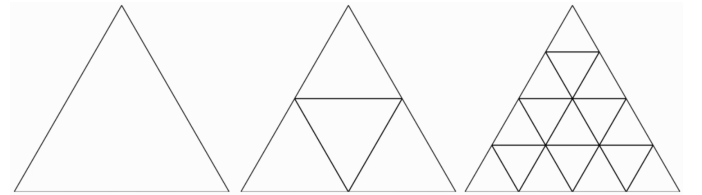


Fig. 9. Illustration of the triangle cutout used in this project.

Now that we have cleared an area where the mesh has been densified, we need to apply a deformation to it so that a scar appears. A simple deformation that produces a sufficiently realistic result is to apply a Gaussian function.

$$f(x) = e^{(-\frac{x}{\sigma})^2}$$

This function takes as its argument the distance between the mesh vertex and our geodesic. For a touch of realism, the closer a point is, the greater the Gaussian deformation will be and the pinker the color will be. Conversely, the mesh will be less and less deformed and colored Figure 10. To obtain this notion of distance, simply project the point onto the segments defined by the outline of our scar and calculate the Euclidean distance between our point and its projection.

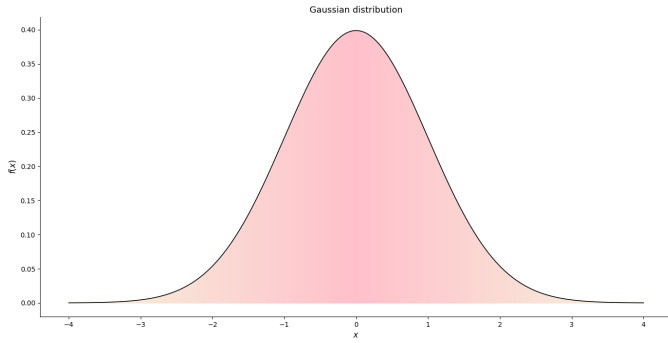


Fig. 10. Illustration of the gaussian deformation.

4 RESULTS

Ultimately, in this project, the processing time for a knee mesh (see Figure 11), with the processing of the 3D object, the creation of a list of neighbors, the creation of the underlying graph, the use of a shortest path algorithm, the various geodesic approximation algorithms, then the densification of the mesh and finally its deformation, the entire process takes about **forty seconds**. This is a reasonable amount of time for visual rendering during a preoperative consultation. However, there is still potential for improvement in terms of optimizing data management and calculation costs.

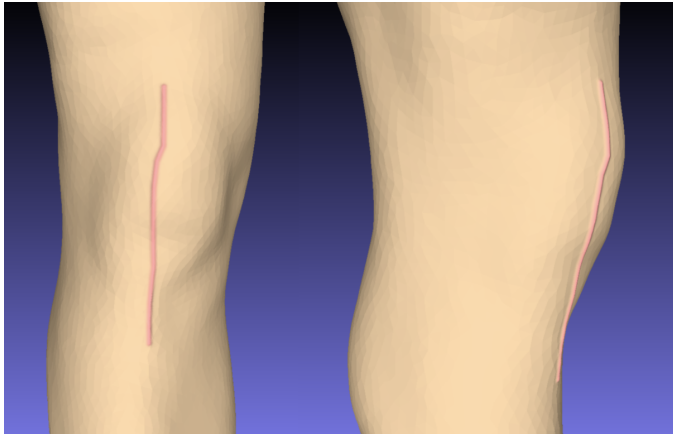


Fig. 11. Visualization of the result in Meshlab, with a knee mesh composed of 21,000 vertices and 18,500 triangles.

5 CONCLUSION

Despite conclusive results, several improvements could be made, this time around in terms of graphics. First, we could optimize the triangular path found. The path given by our shortest path algorithm may not be optimal in terms of geodesics, because the underlying graph is constructed from the centroids of the triangles. It might therefore be wise to use the vertices of the triangles directly as vertices of the graph. It would also be possible to recalculate the shortest paths on sub-parts of the path when it is not optimal. In other words, when using algorithms where we move our points along the edges of our triangles, when they reach the end of the segment, this may mean that the path would be more relevant with the adjacent triangle. It would also be possible to perform a gradient descent

to approximate our geodesic, instead of using an iterative algorithm.

Another interesting approach to geodesic calculation was proposed by **Sharp & Crane (2020)** [3]. This method is an original technique that simply consists of “flipping” the edges of the mesh, a bit like rearranging the triangles to gradually straighten the path. With each flip, the curve becomes straighter and shorter, until it converges to an exact geodesic. This approach is very fast (a few milliseconds, even for very large meshes). This method could be another alternative and would require further study.

REFERENCES

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] N. Sharp and K. Crane, “You can find geodesic paths in triangle meshes by just flipping edges,” *ACM Trans. Graph.*, vol. 39, no. 6, 2020.